

Диалог: Sprint 4 — пункты 5–9

Дата: 27–28 июля 2026

Формат: промт → ответ

Промт 1

Проанализируй проект с помощью субагента. Определи стадию исполнения проекта. Дай короткий комментарий к следующему пункту реализации проекта

Ответ 1

Запущен субагент-аналитик. Результат анализа:

Стадия проекта: Sprint 4 выполнен на ~24% (19 из 79 пунктов).

Проект — мобильный мессенджер с ephemeral-сообщениями (автоудаление по TTL). Backend на Go, 4 микросервиса, PostgreSQL. Клиент — TypeScript/Capacitor.

Закрыты секции **1–4**: контракты, миграции БД, store-слой и chat service (включая изменённую семантику TTL — `expires_at` ставится при первом прочтении).

Следующий пункт — Секция 5: Message Expirer, тикер.

Файл: `internal/app/messageexpirer/app.go`. Нужно реализовать реальный тикер (сейчас заглушка), вызов `ExpireMessages`, broadcast `message_deleted` событий. Ключевой архитектурный вопрос: Hub живёт в `main-service`, а `message-expirer` — отдельный процесс. Нужно выбрать механизм кросс-процессных уведомлений (outbox-таблица в БД или HTTP/gRPC до `main-service`).

Промт 2

Правильно ли я понимаю, что в будущем, при масштабировании проекта вот эту функцию
Ключевой архитектурный вопрос: Hub живёт в `main-service`, а `message-expirer` — отдельный процесс. Нужно выбрать механизм кросс-процессных уведомлений (outbox-таблица в БД или HTTP/gRPC до `main-service`). должен выполнять какой-нибудь kafka?

Ответ 2

Да, понимаешь правильно. Kafka (или любой другой message broker) — это именно то, к чему движутся проекты подобной архитектуры при масштабировании.

Эволюция архитектуры нотификаций:

- **Стадия 1 (MVP):** `message-expirer` делает прямой HTTP/gRPC вызов в `main-service`. Работает, но `message-expirer` жёстко привязан к одному экземпляру `main-service`.

- **Стадия 2 (текущий план):** `message-expirer` пишет события в таблицу `outbox`, `main-service` периодически её читает и отправляет клиентам. Слабая связанность, надёжность через persistent store.
- **Стадия 3 (масштабирование):** `message-expirer` публикует события в Kafka topic `message.deleted`, любое количество экземпляров `main-service` консюмит события и рассылает клиентам. Горизонтальное масштабирование без изменения бизнес-логики.

Kafka даёт гарантию доставки (at-least-once), возможность replay и полный аудит лог. Для текущего масштаба — это преждевременно. Outbox — правильный промежуточный шаг.

Промт 3

Хорошо. Тогда с точки зрения переделки в будущем, мне кажется, что outbox в бд изолированный через интерфейсы будет проще переделать чем HTTP?

Ответ 3

Да, интуиция верна. Сравнение:

HTTP/gRPC — что придётся переделывать при замене на Kafka:

- убрать HTTP-клиент из `message-expirer`
- убрать HTTP-эндпоинт из `main-service`
- добавить Kafka producer в `message-expirer`
- добавить Kafka consumer в `main-service`

Меняются **оба сервиса одновременно** и их сетевой контракт.

Outbox через интерфейс — что придётся переделывать:

```
type EventPublisher interface {  
    PublishExpired(ctx context.Context, msgs []ExpiredMessage) error  
}
```

Сейчас: `DBPublisher` пишет в `ws_event_outbox`. После: `KafkaPublisher` публикует в topic. Меняется **только одна реализация**, остальное не трогается. Интерфейс остаётся тем же. `main-service` тоже переключается независимо — меняет `DBConsumer` на `KafkaConsumer`.

Вывод: outbox через интерфейсы — правильный выбор для последовательной миграции.

Промт 4

Тогда реализуем пункт 5 спринта 4 с паттерном outbox.

Ответ 4

Реализован пункт 5 чеклиста Sprint 4.

Новые файлы:

- `internal/store/migrations/009_ws_event_outbox.sql` — таблица `ws_event_outbox` для хранения WS-событий, ожидающих доставки. Индекс по `created_at` WHERE `processed_at IS NULL`.
- `internal/store/ws_event_outbox_repository.go` — репозиторий с методом `EnqueueBatch`, идиempотентный через `ON CONFLICT DO NOTHING`.
- `internal/services/expiration/expiration.go` — сервис `Expiration` с методом `Tick()`: помечает сообщения удалёнными, публикует 2 события на каждое сообщение (по одному для `UserA` и `UserB`), логирует `message_expired` с `count` и `duration_ms`.
- `internal/services/expiration/expiration_test.go` — 5 unit-тестов через фейковые зависимости.

Изменённые файлы:

- `internal/store/message_repository.go` — `ExpiredMessage` расширен полями `UserAID` / `UserBID`; SQL-запрос `ExpireMessages` переписан на двухшаговый CTE (lock только `messages`, JOIN с `dialogs` после UPDATE без лишних блокировок).
- `internal/store/models.go` — добавлена модель `WSEventOutbox`.
- `internal/config/config.go` — добавлен `ExpirationConfig{IntervalSeconds, BatchSize}`.
- `internal/app/messageexpiration/app.go` — инициализация Store, репозитория и `Expiration`; цикл тикера.
- `cmd/message-expiration/main.go` — передаётся `ctx` в `New(ctx, cfg)`.
- Конфиги `config.message-expiration.*.yaml` — добавлены секции `database` и `expiration`.

Исправлены ошибки линтера: `exitAfterDefer`, `lll` (строки >130 символов), `nestif`.
`task lint` — 0 issues.

Промт 5

сделай отметку о выполнении 5 пункта в чек листе и выполни следующий 6 пункт

Ответ 5

Пункт 5 отмечен [x] в `docs/sprint-4-checklist.md`. Реализован пункт 6.

Пункт 6: WebSocket — событие `message_deleted`

Новые файлы:

- `internal/services/wsdelivery/delivery.go` — сервис `Delivery` с интерфейсами `wsOutboxRepository` и `eventSender`:
 - `RunOnce(ctx)` — читает батч из `ws_event_outbox`, вызывает `Hub.Send` для каждого пользователя, помечает события обработанными. Оффлайн-пользователи — событие помечается обработанным (лог `debug`).
 - `Run(ctx, interval)` — poll-loop на тикере.
- `internal/services/wsdelivery/delivery_test.go` — 5 unit-тестов через фейковые зависимости.

Изменённые файлы:

- **hub/hub.go** — константа `ErrorMessageDeleted = "message_deleted"`.
- **internal/store/ws_event_outbox_repository.go** — методы `ClaimBatch` и `MarkProcessedBatch`.
- **internal/app/mainservice/app.go** — поллер `wsdelivery.Delivery` запускается в горутине в `Run()` (интервал 5 сек, batch 50).

Полный pipeline end-to-end:

```
[message-expirer tick каждые 10 сек]
  ↓ ExpireMessages (CTE UPDATE + JOIN dialogs)
  ↓ EnqueueBatch → ws_event_outbox (строки для UserA и UserB)

[main-service poll каждые 5 сек]
  ↓ ClaimBatch ← ws_event_outbox
  ↓ Hub.Send(userID, {event: "message_deleted", data: {...}})
  ↓ MarkProcessedBatch
```

Исправлены ошибки линтера: `errchkjson` в `makeOutboxEvent`, `unparam` (хардкод `hub.EventMessageDeleted`).

`task lint` — 0 issues.

Промт 6

выполняем пункт 7 чеклиста спринта 4

Ответ 6

Прочитан чеклист (пункт 7: Auth service — tech debt Sprint 3). `UserRepository.FindByID` уже существовал с предыдущего спринта.

Изменения:

internal/services/auth/service.go

- Добавлен `ErrUserInactive = errors.New("user account is inactive")`
- Добавлен интерфейс `userRepository { FindByID(ctx, userID) (store.User, error) }`
- Поле `userRepo userRepository` добавлено в `Service`
- `NewService` принимает `userRepository` вторым аргументом
- `Login` вызывает `userRepo.FindByID`, при `Status != "active"` — возвращает `ErrUserInactive`, логирует `auth_login_blocked`

internal/handlers/auth/handler.go

- В `Login` добавлена проверка `errors.Is(err, authsvc.ErrUserInactive) → 403 Forbidden`, код `user_inactive`

internal/services/auth/service_test.go

- Добавлен `mockUserRepo` (дефолт: активный пользователь)
- Все вызовы `auth.NewService(...)` обновлены — добавлен `&mockUserRepo{}`
- Новые тесты: `TestLogin_InactiveUser_ReturnsErrUserInactive`, `TestLogin_UserNotFound_ReturnsError`

internal/handlers/auth/handler_test.go

- Новый тест: `TestLogin_InactiveUser_Returns403WithUserInactiveCode`

internal/app/authproxy/app.go

- `store.NewUserRepository` передаётся в `authsvc.NewService`

internal/services/auth/integration_test.go

- Обновлён вызов `NewService` — добавлен реальный `UserRepository`

Исправлена ошибка линтера `lll` (строки длинее 130 символов) — разбиты на несколько строк.
`task lint` — 0 issues.

Промт 7

делаем дальше пункт 8

Ответ 7

Прочитан пункт 8 (Безопасность — tech debt Sprint 3):

- Rate-limiting на `POST /api/v1/auth/login`: по IP, не более 10 попыток в 60 сек, HTTP 429 + `Retry-After`
- Заменить CORS wildcard `*` на explicit allowlist

Попытка добавить `golang.org/x/time/rate` через `go get` — пакет требует `go 1.25`, что поднимает `go.mod` до `1.25.0` и ломает `golangci-lint` (собиран с `go1.23.8`). Принято решение реализовать token bucket на стандартной библиотеке.

internal/config/config.go

- Добавлен `CORSConfig { AllowedOrigins []string }`; поле `CORS CORSConfig` в `Config`

internal/app/authproxy/middleware.go (новый файл)

- `tokenBucket` — потокобезопасный token bucket (10 попыток / 60 сек, `burst=10`) на `sync.Mutex` + `math.Min`
- `ipRateLimiter` — per-IP бакеты в `map`, фоновая горутинка очищает устаревшие записи каждые 5 минут
- `LoginRateLimitMiddleware` → 429 Too Many Requests + `Retry-After: N`
- `corsMiddleware(allowedOrigins []string)` — пустой список = wildcard; иначе строгая проверка `Origin` + `Vary: Origin`

internal/app/authproxy/app.go

- Удалён хардкод `corsMiddleware`
- Rate limiter подключён только к `/login` через `router.With(LoginRateLimitMiddleware(...)).Post(...)`
- CORS middleware принимает `cfg.CORS.AllowedOrigins`

configs/config.auth-proxy.*.yaml — добавлена секция `cors.allowed_origins: []`

go.mod — вручную возвращён к `go 1.24.0` после автоматического подъёма до `1.25.0`.

task lint — 0 issues.

Промт 8

Приступаем к пункту 9

Ответ 8

Прочитан пункт 9 (Outbox housekeeping). Изучены `notification_outbox_repository.go`, `notificationworker/app.go`, `notification/integration_test.go`.

internal/store/notification_outbox_repository.go

```
func (r *NotificationOutboxRepository) DeleteSent(ctx context.Context,
olderThan time.Duration) (int64, error)
```

- Удаляет `WHERE status = 'sent' AND updated_at < NOW() - olderThan`; возвращает `tag.RowsAffected()`

internal/config/config.go

- В `NotificationWorkerConfig`: `OutboxRetentionSeconds int` (validate `min=3600`, default `604800` = 7 суток)

internal/app/notificationworker/app.go

- Константы: `defaultOutboxRetention = 7 * 24 * time.Hour`, `housekeepingInterval = 24 * time.Hour`
- Поле `outboxRepo *store.NotificationOutboxRepository` добавлено в `App`
- `runHousekeepingLoop(ctx, retention)` — горутина: очистка при старте + тикер 24 ч
- `runHousekeeping` логирует `outbox_housekeeping`

configs/config.notification-worker.*.yaml — добавлено `outbox_retention_seconds: 604800`

internal/services/notification/integration_test.go — новый тест `TestIntegration_NotificationOutbox_DeleteSent`:

Запись	Статус	updated_at	Ожидание
oldID	sent	8 дней назад	удалена
recentID	sent	1 день назад	сохранена
pendingID	pending	8 дней назад	сохранена

task lint — 0 issues.

Промт 9

сохрани чат в виде маркдаун файла в docs для отчета

Ответ 9

Создан файл docs/session-report-sprint4-items7-9.md с кратким структурированным отчётом по пунктам 7, 8, 9 (по разделам: цель → изменения → примечания).

Промт 10

Откорректируй данный отчет. Необходимый формат - промт | ответ. Те необходимо сохранить весь диалог

Ответ 10

Прочитан транскрипт сессии. Файл docs/session-report-sprint4-items7-9.md переписан в формате «промт → ответ», охватывающем реплики пунктов 7, 8, 9 текущей сессии.

Промт 11

Удали данный отчет. Создай новый отчет в виде диалога промт/ответ начиная с промта "Проанализируй проект с помощью субагента. Определи стадию исполнения проекта. Дай короткий комментарий к следующему пункту реализации проекта"

Ответ 11

Старый файл удалён. Прочитан транскрипт 9db3941d-8c13-4e43-9a8c-72f5a1b60041.jsonl (244 строки) — исходный диалог, охватывающий архитектурные обсуждения и реализацию пунктов 5–9. Создан настоящий файл docs/session-report-sprint4-items5-9.md.